Operating System Security

Mobile OS Security

Mobile OS Security

We'll focus on Android as it has the highest global market share (~74% in 2019 [2]).

Source for today's lecture:

- [1] "Android Security Overview", http://source.android.com/devices/tech/security/index.html, last accessed March 13, 2026.

- [2] "iPhone vs Android market share", https://www.macworld.co.uk/feature/iphone/iphone-vs-android-market-share-3691861/, last accessed March 13, 2025.

Android

Android is an application execution environment for mobile devices (by Google).

It includes an OS, an application framework, and core apps.

- OS: Linux kernel (for device drivers, memory & process management, networking, etc.)
- Android native libraries: written in C/C++. Incorporated into Android applications using Java Native Interfaces (JNI). Used by various system components.
- Android run-time: compatible with predecessor Dalvik runtime. Also contains core libraries.

Android (continued)

Android runtime (ART): runs .dex files that are designed to be more compact and memory-efficient than Java class files.

Core libraries: written in Java.

Framework layer: written in Java. Includes Google-provided tools as well as proprietary extensions and services.

Application layer: provides apps such as a phone, web browser, email client, etc.

Android Software Stack

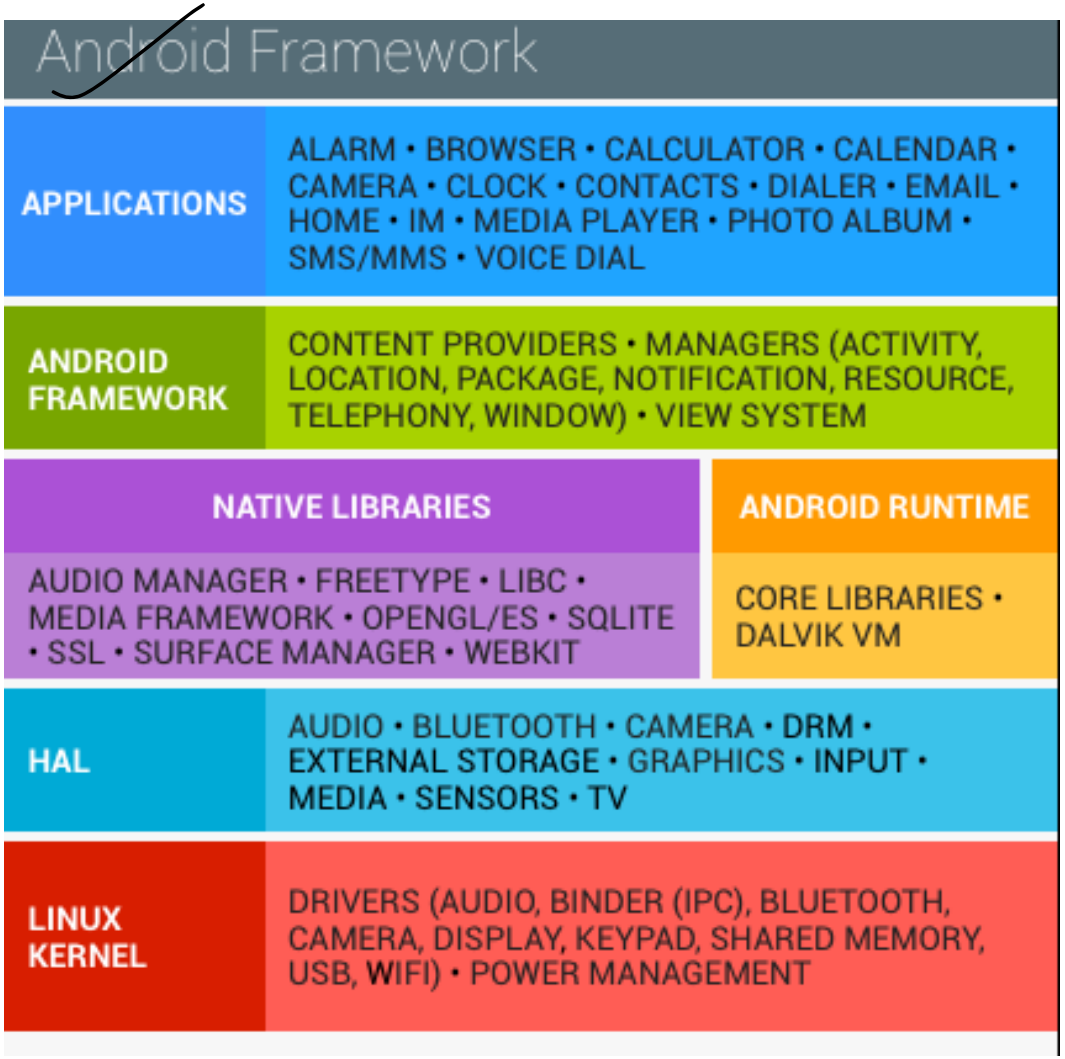


*Image from [1], Fig. 1.

Android Apps

Each app is packaged in an .apk archive for installation.

- Similar to a .jar file (used in Java) in that it holds all code and non-code resources (e.g., images) for the app.

Apps are normally written in Java (or Kotlin) based on APIs provided by the Android SDK.

Some core security services

Google Play: A collection of services that allow users to discover, install, and purchase apps. Provides e.g.:

- Community review
- Application security scanning

Android Updates: provides patches for security and functionality

Some other core security services

App services: Frameworks that allow apps to use cloud capabilities such as backing up app data.

Verify Apps: Warn or automatically block the installation of harmful apps, and continually scan apps on the device, warning about or removing harmful apps.

SafetyNet: A privacy preserving intrusion detection system to assist Google tracking, mitigate known security threats, and identify new security threats.

Android Device Manager: A web app and Android app to locate lost or stolen device.

Android Security Mechanisms (1/)

Linux Mechanisms	Description	Security Notes
POSIX users	Each application is associated with a unique user ID (or UID). Exception: sharing possible in apps created by the same developer.	Creates a "sandbox" for each application above the kernel (including os libraries, runtime, and app framework)
File access	Subject to regular Linux permissions. The application's directory is only available to the application.	Prevents one application from accessing another's files

The Application Sandbox

Will it always prevent an application's files from being accessed by another application?

Is it as good as Mandatory Access Control by SELinux?

(Can you think of a proof by counter-example?)

- Hint: what mistakes can a developer make?

Android Security Mechanisms (2/)

Environmental features	Description	Security Notes
Type safety	Type safety enforces variable content to adhere to a specific format, both in compiling time and runtime.	Prevents buffer overflows and stack smashing
Application permissions	Each app declares which permission it requires at install time (after 6.0, it can be at first use). **Requires user's approval.**	Limits application abilities to perform malicious behavior

Android Permissions Prompt

Users can choose what permissions to give an app.

It can be modified after installation.

But how many people are careful with them?

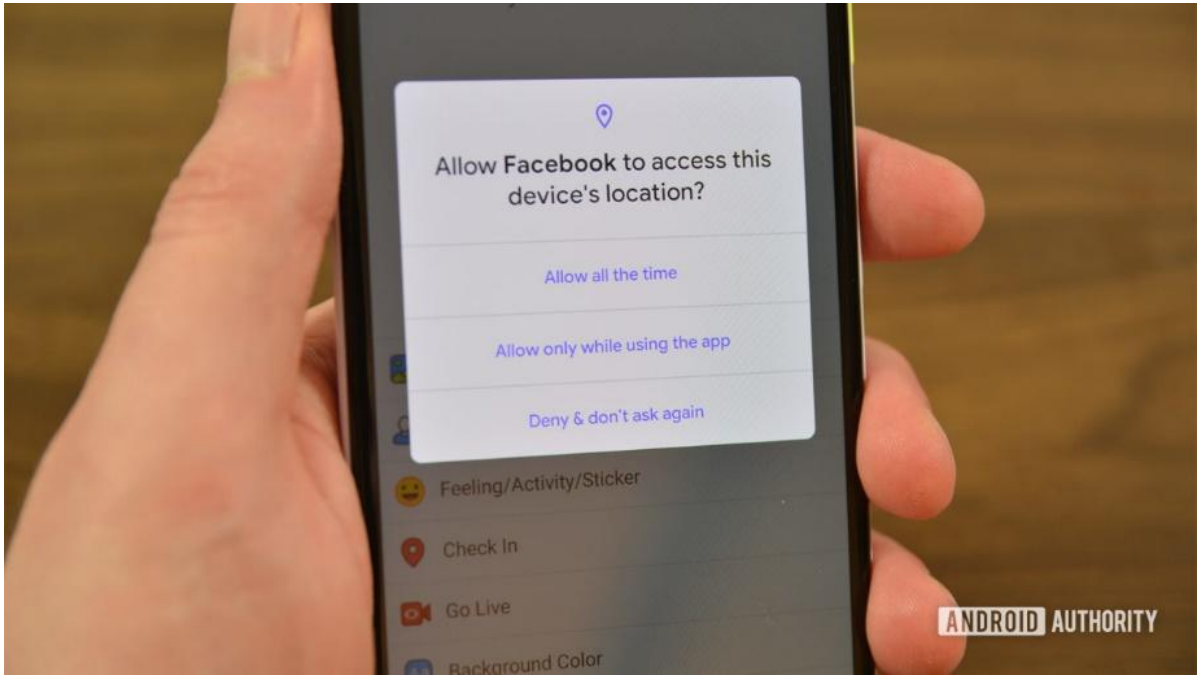


*Image from Joe Hindy, “Android 10 permissions: What’s new and how to use them!”, 2019, https://www.androidauthority.com/android-10-permissions-1040121/ , last accessed March 13, 2025.

Android Security Mechanisms (3/)

Android-specific mechanisms	Description	Security Notes
Component encapsulation	Each component in an application has a visibility level that regulates access to it from other apps (e.g., binding to a service)	Prevents one app from disturbing another, or accessing private components or APIs

Android Security Mechanisms (4/)

Android-specific mechanisms	Description	Security Notes
Signing applications	The developer signs application .apk files, and the package manager verifies them.	Verifies that two apps are from same source. Can be using a self-signed certificate.
Per-app virtual machine	Each application runs in its own virtual machine (unless signed by same certificate)	Prevents buffer overflows, remote code execution, and stack smashing.

Android Security Program – Key Components

Design review. Security controls are integrated into the architecture of the system.

Pen testing and code review. During development, components are subject to vigorous security reviews by the Android Security Team, Google's Information Security Engineering Team, and independent security consultants.

Open source and community review. It's open source so anyone interested can review it.

Incident response. Constant monitoring and aim for quick push of updates, removing apps from devices, etc.

How large is the TCB?

Approx. 20 million lines of code!

The Linux kernel has about 10 million lines.

Memory Management Security Enhancements

- Many additional libraries used to prevent stack, heap, and integer overflows, as well as format string vulnerabilities.
- DEP --- prevents code execution on stack and heap.
- Address Space Layout Randomization (ASLR)

More About Data Execution Prevention (DEP)

** Note: most modern OSes implement this.

Makes it difficult to exploit a memory vulnerability, by preventing code execution from data pages

Rationale: typically, code is executed from code pages

Marks all memory locations in a process (e.g., stack and heap) as non-executable (NX) unless the location explicitly contains executable code

Processors that support this can raise an exception when code is executed from a page that is marked as NX

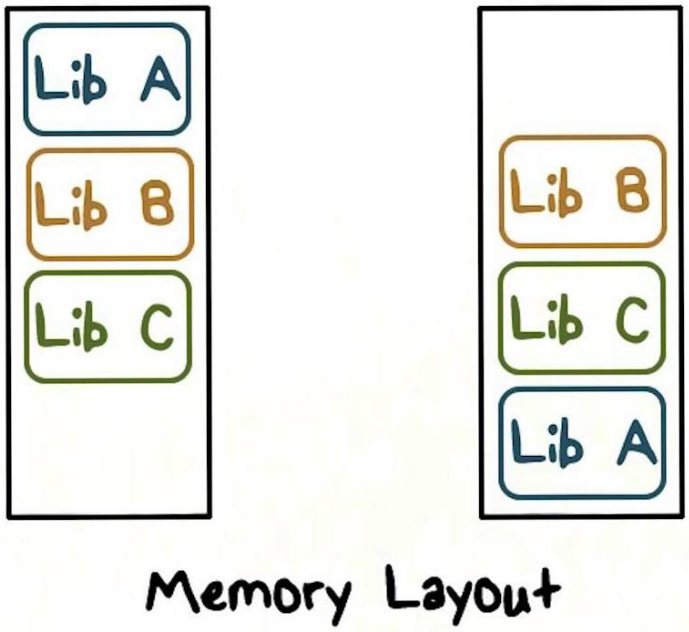
Address Space Layout Randomization (ASLR)

** Note: most modern OSes implement this.

Makes DEP defeats (e.g., "return oriented programming") much harder.

Randomizes where processes and libraries are loaded into memory

- Makes exploitation of memory errors much more difficult.



Cryptographic options

5.0 and later supports full-disk encryption.

- Full-disk encryption uses a single key—protected with the user's device password. At boot, users must provide their credentials before any part of the disk is accessible.

7.0 and later supports file-based encryption.

- File-based encryption allows different files to be encrypted with different keys that can be unlocked independently.

SELinux on Android

It exists since version 4.3, but in 4.3 was in permissive mode by default.

By default is in enforcing mode from version 4.4, but some domains in permissive mode.

From version 5.0, everything is in enforcing

Has its own default policy that works for Android (the targeted policy would not make sense in the Android context!).

Discussion Question

Given Android's security mechanisms, can you think of a threat that might still exist on Android, without SELinux enabled?

- Consider developers and the mistakes they can make!

SELinux on Android - Reference

Primary source for this material:

- [3] S. Smalley and R. Craig. "Security Enhanced (SE) Android: Bringing Flexible MAC to Android." Network & Distributed System Security Symposium (NDSS). 2013.

SE Android

A reference implementation for how to implement SELinux on Android.

Demonstrates the value provided by SELinux in confining various root exploits and application vulnerabilities

Android Weaknesses (sans SELinux)

The ability of flawed or malicious apps to leak access to data

The inability to confine any system daemons or setuid programs that run with the root or superuser identity

Course grained privileges

SE Android Policy Goals

Confine the privileged daemons in Android

Ensure that the Android middleware components cannot be bypassed

Strongly isolate apps from each other and from the system

SE Android Policy Created

Created a small policy from scratch tailored to Android's userspace software stack and to our security goals.

The Type Enforcement (TE) portion of the policy was configured to define confined domains for the system daemons and apps.

The Multi-Level Security (MLS) portion of the policy was configured to isolate app processes and files from each other based on MLS categories.

This approach yielded a small, fixed policy at the kernel layer with no requirement for policy writing by Android app developers.

Comparing SE Android Policy to Fedora's

	SE Android	Fedora
Size	71K	4828K
Domains	39	702
Types	182	3197
Allows	1251	96010
Transitions	65	14963
Unconfined	3	61

Table 3. Policy size and complexity.

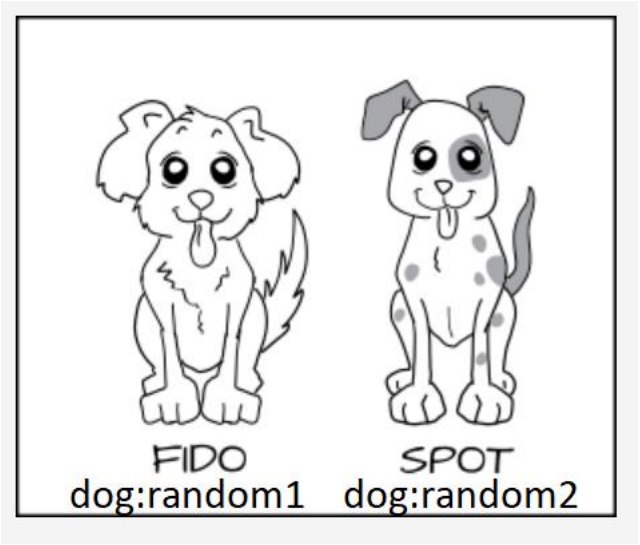
Automatic per app policies

Managing access control policies for potentially hundreds of apps on a per-app basis is not feasible

In order to express app-permission authorizations without having to specify per-app rules, they used a method based on X.509 certificates.

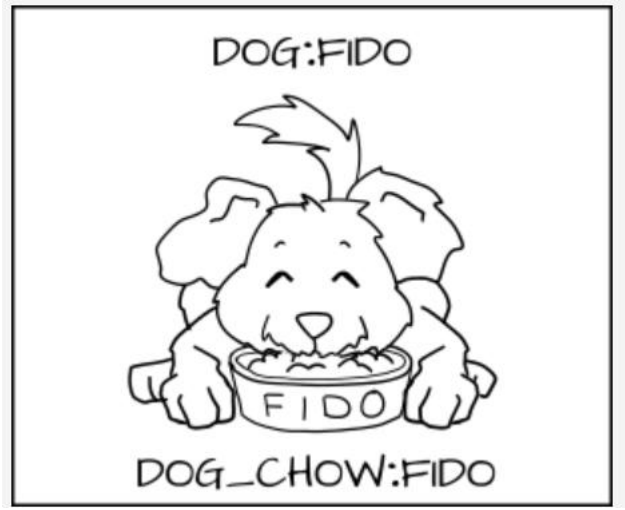
- Android already requires each installed app to be signed with such a certificate.
- They leveraged this existing attribute of Android apps by identifying groups of apps by their certificate.

Let's go back to our cartoon again

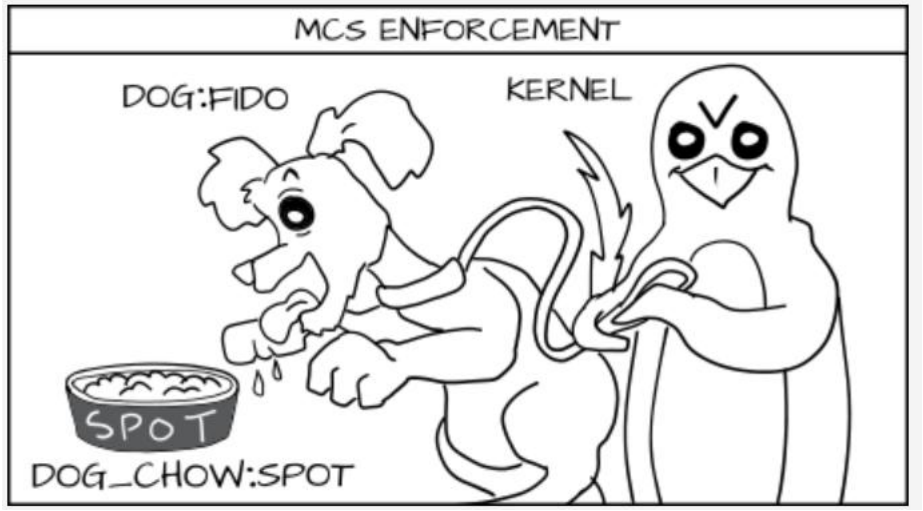


Cartoons from: https://opensource.com/business/13/11/selinux-policy-guide (last accessed March 13, 2025)

Let's go back to our cartoon again (continued)



- Analogy is that:
  - Dogs=apps
  - Dog food = files and resources needed by the app
  - Each app has its own random ID (a category).



OK, so what does SE Android prevent?

Root exploit examples:

- GingerBreak and Exploid
- Others (see paper [3] if interested)

Application vulnerability examples:

- Skype
- Opera Mobile
- Others (see paper [3] if interested)

Root exploits

These exploits escalate privilege from an unprivileged app to gain root access to the device.

Often these root exploits are developed by the Android modding community for the purpose of enabling them to modify their own devices for customization and optimization.

However, these exploits can also be leveraged by malware to gain complete control of a user's device.

GingerBreak - Background on vold

vold (the Android volume daemon) is a system service that runs as root on Android.

It manages the mounting of the SDcard and the mounting of encrypted storage.

It listens for messages on a netlink socket in order to receive notifications from the kernel.

Had a vulnerability in handling netlink messages:

- did not verify they originated from the kernel, so possible for unprivileged apps to send msgs.
- used a signed integer from the message as an array index, and did not check if negative!

GingerBreak: How it works (1/)

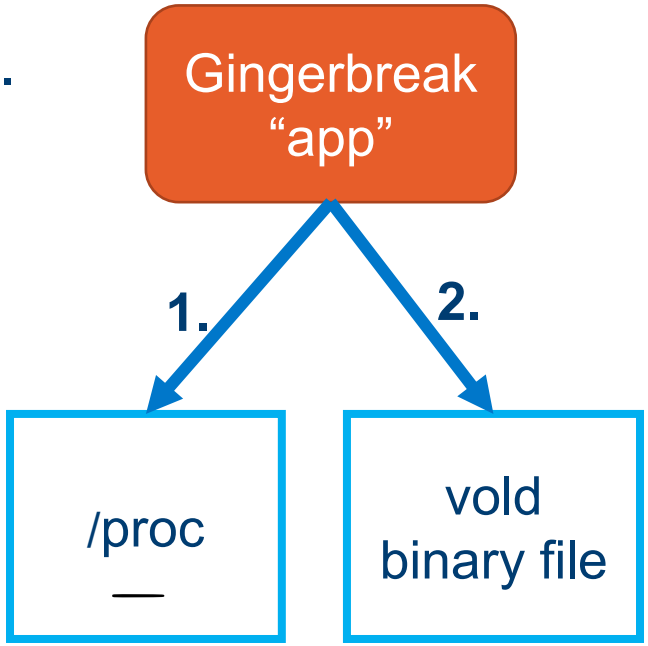
Gingerbreak took advantage of the vold vulnerability and enabled root access from a user shell.

- Used in the GingerMaster malware.

How it works:

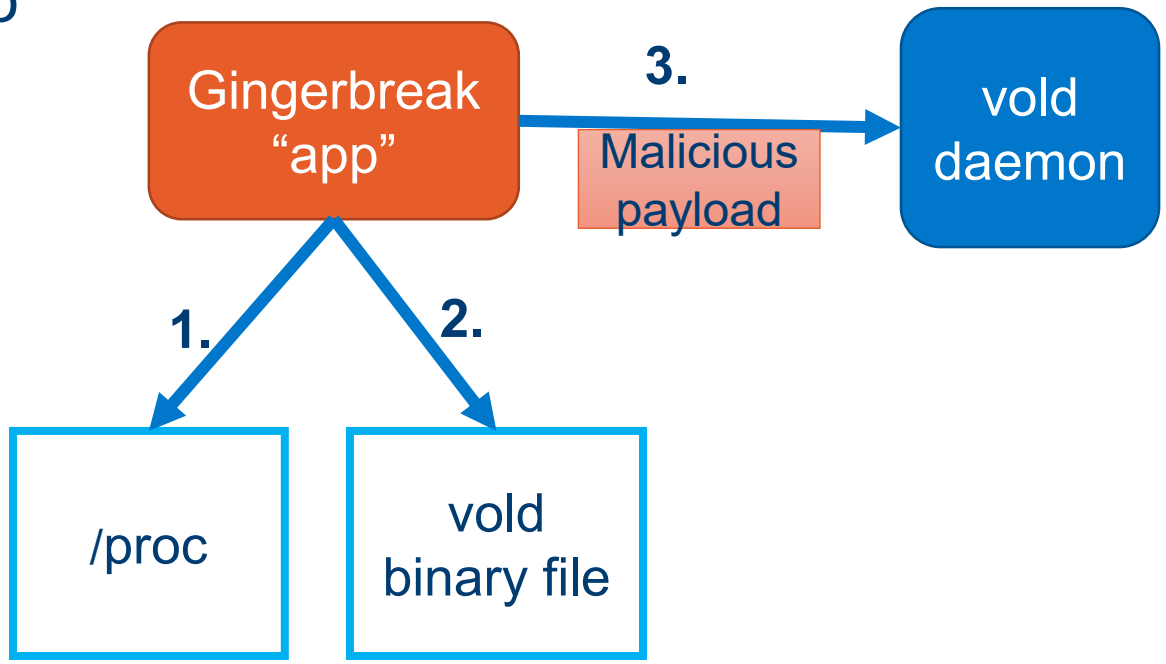
1. Obtains the PID of the vold process from the proc filesystem.

2. Obtains information required to craft the malicious payload by reading several files from the system partition (e.g., vold binary file).



GingerBreak (2/)

3. Sends the malicious payload to vold via a netlink socket.

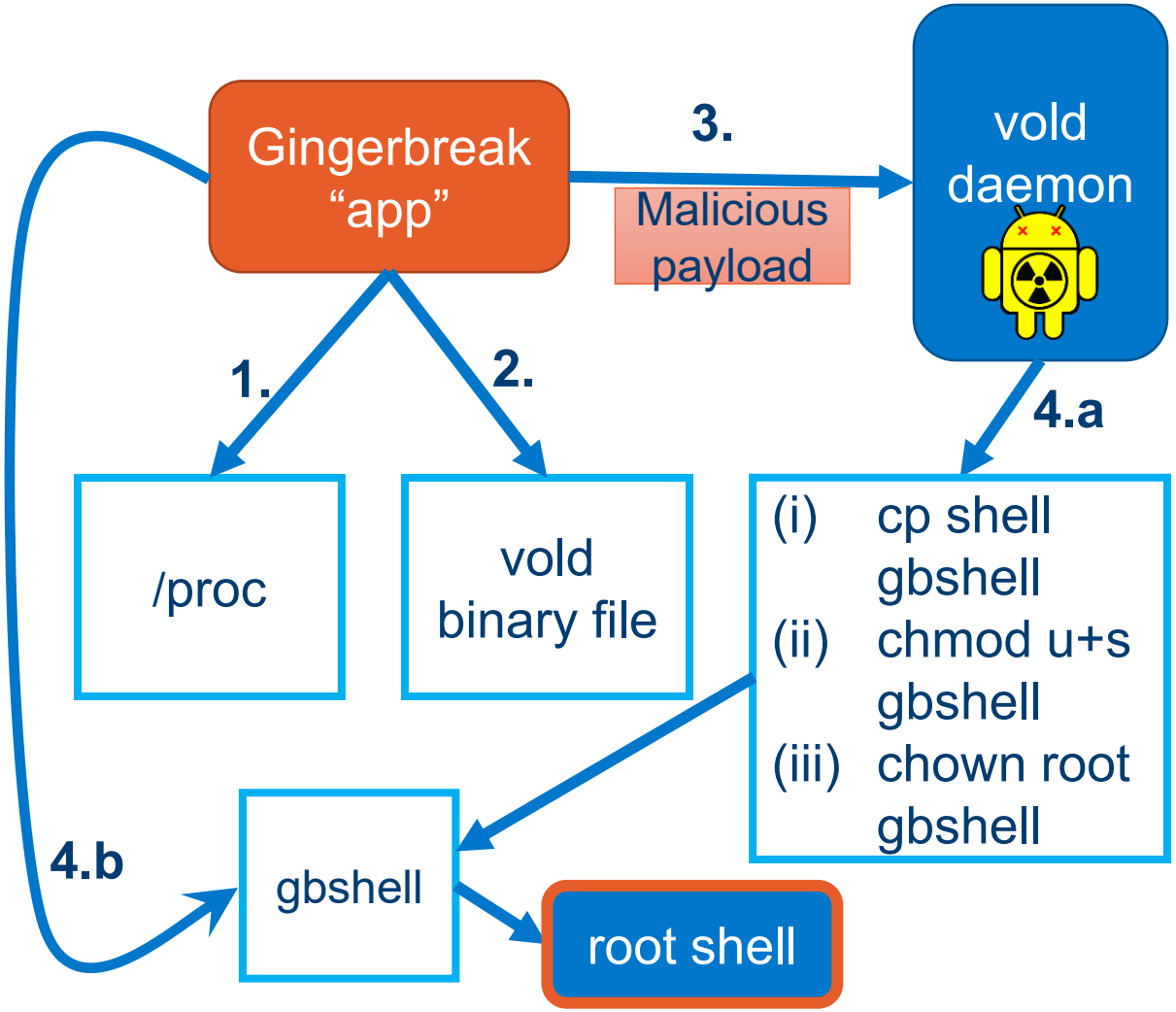


GingerBreak (3/)

4. Payload triggers execution of the exploit by vold, (which is then running with root privileges).

a) Next it creates a setuid-root shell

b) GingerBreak's process executes this shell to give the user a root shell.

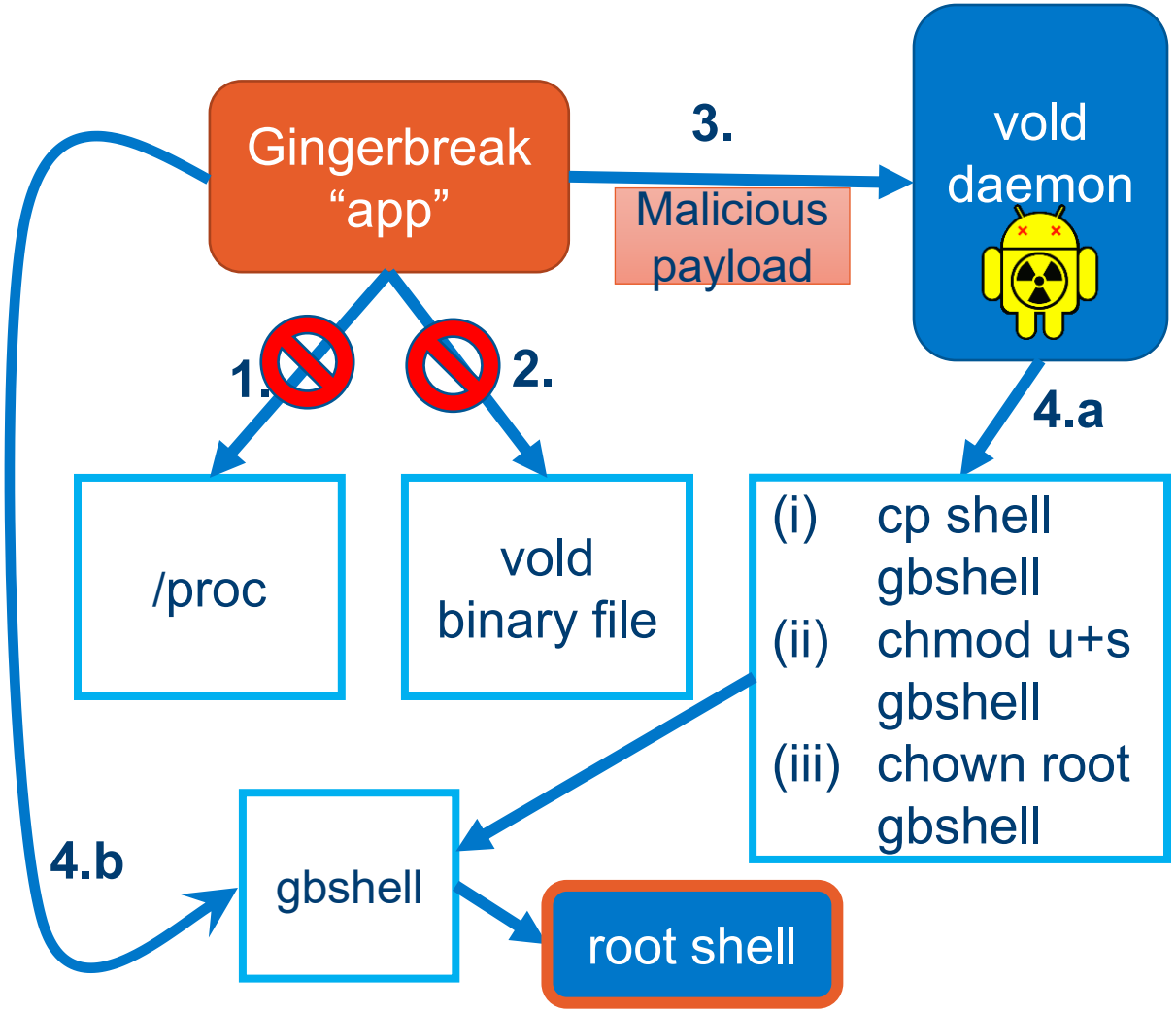


GingerBreak (4/)

Also uses access to the Android logging facility (logcat) in order to dynamically observe the effect of its attacks on vold and refine it incrementally until it succeeds.

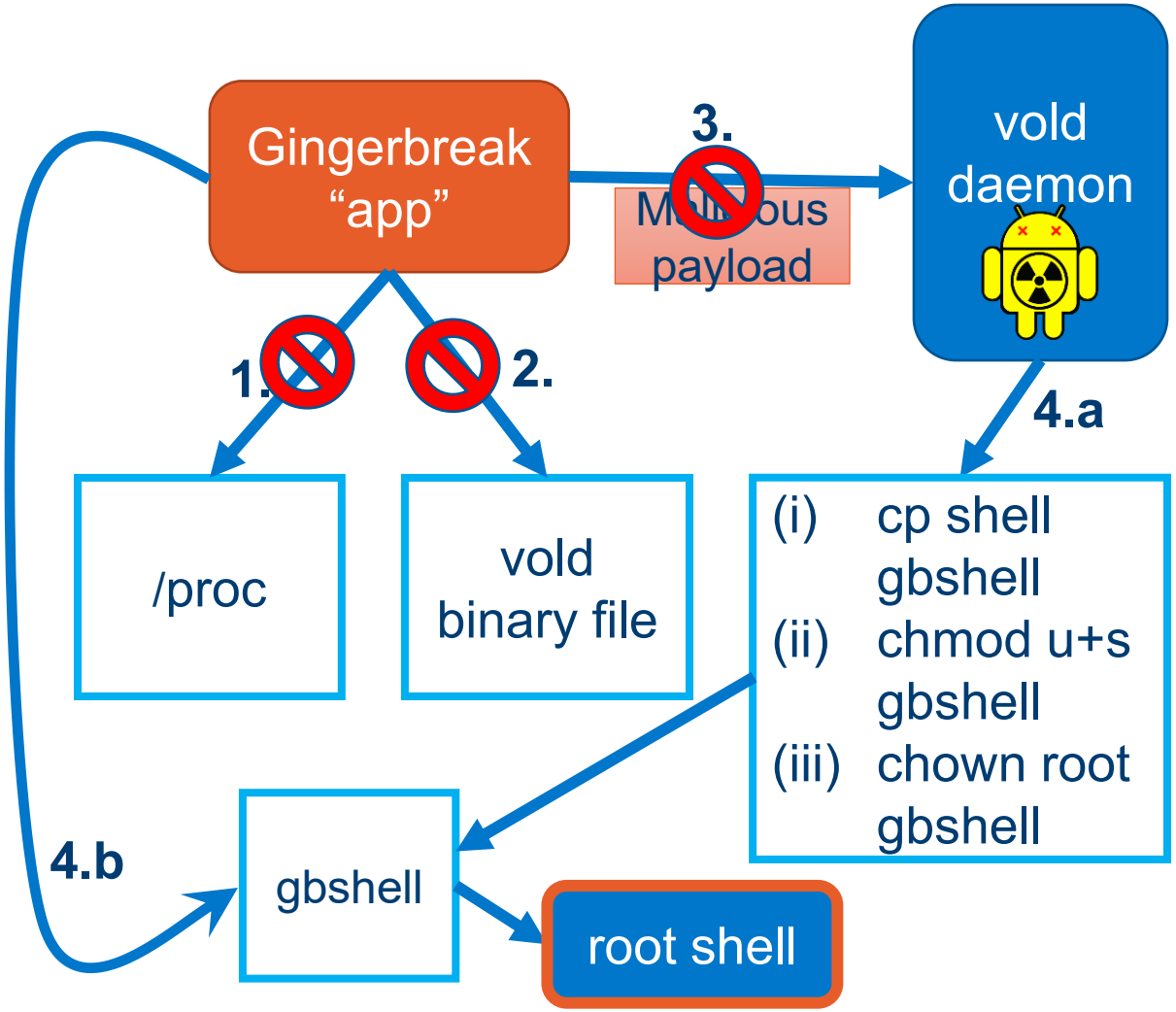
GingerBreak on SE Android (1/)

- The policy denied the exploit's attempt to read the proc information for the vold process.
- Also denied the exploit's attempt to read the vold binary to discover the target address range.



GingerBreak on SE Android (2/)

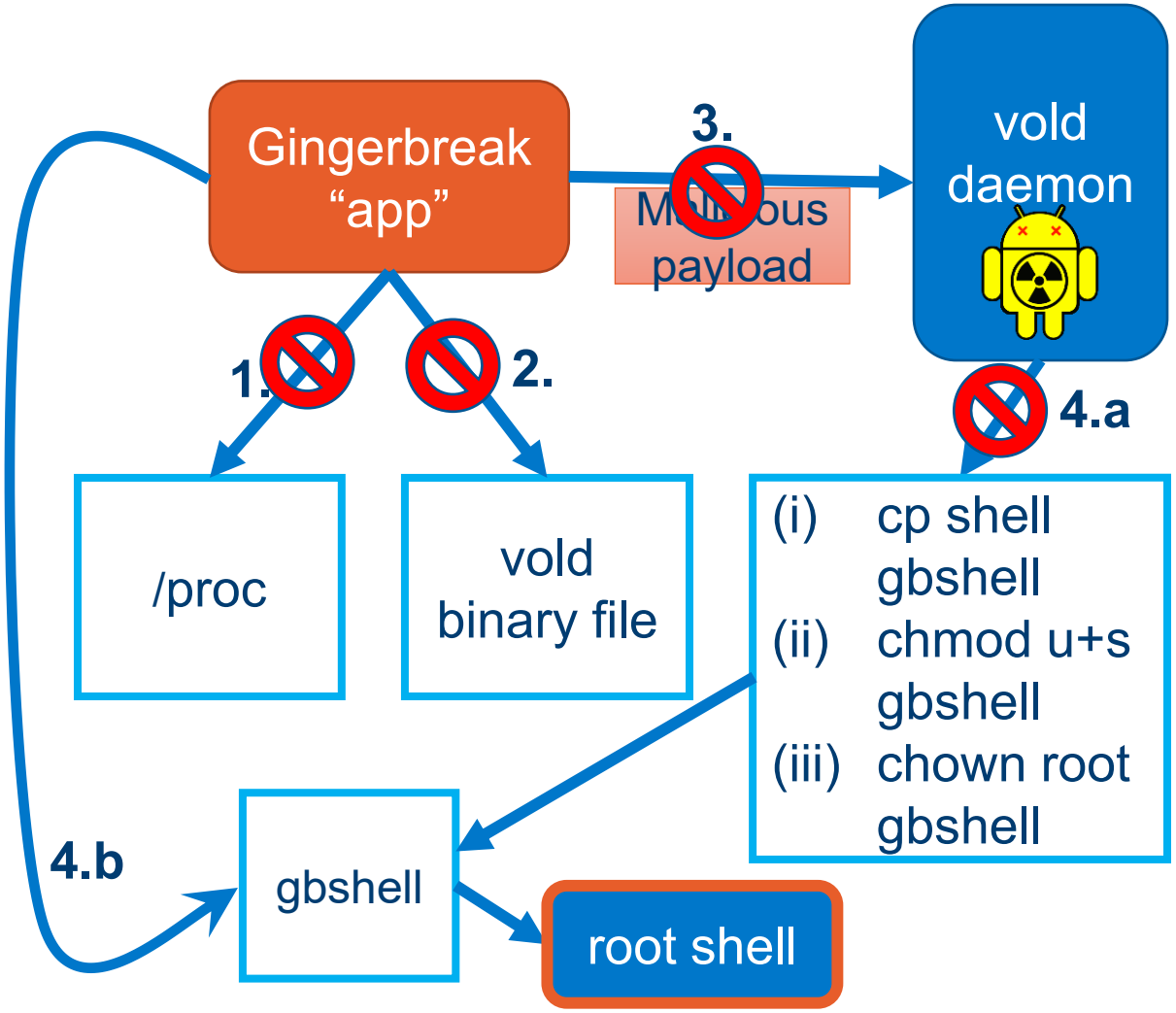
- Even if we assume it made it past 1&2, the policy denies 3, Gingerbreak's attempt to create a netlink socket.
- No legitimate need for user apps to create netlink sockets.
- So the exploit could never reach the vulnerability in vold.



GingerBreak on SE Android (3/)

What would happen next if 1,2,&3 allowed?

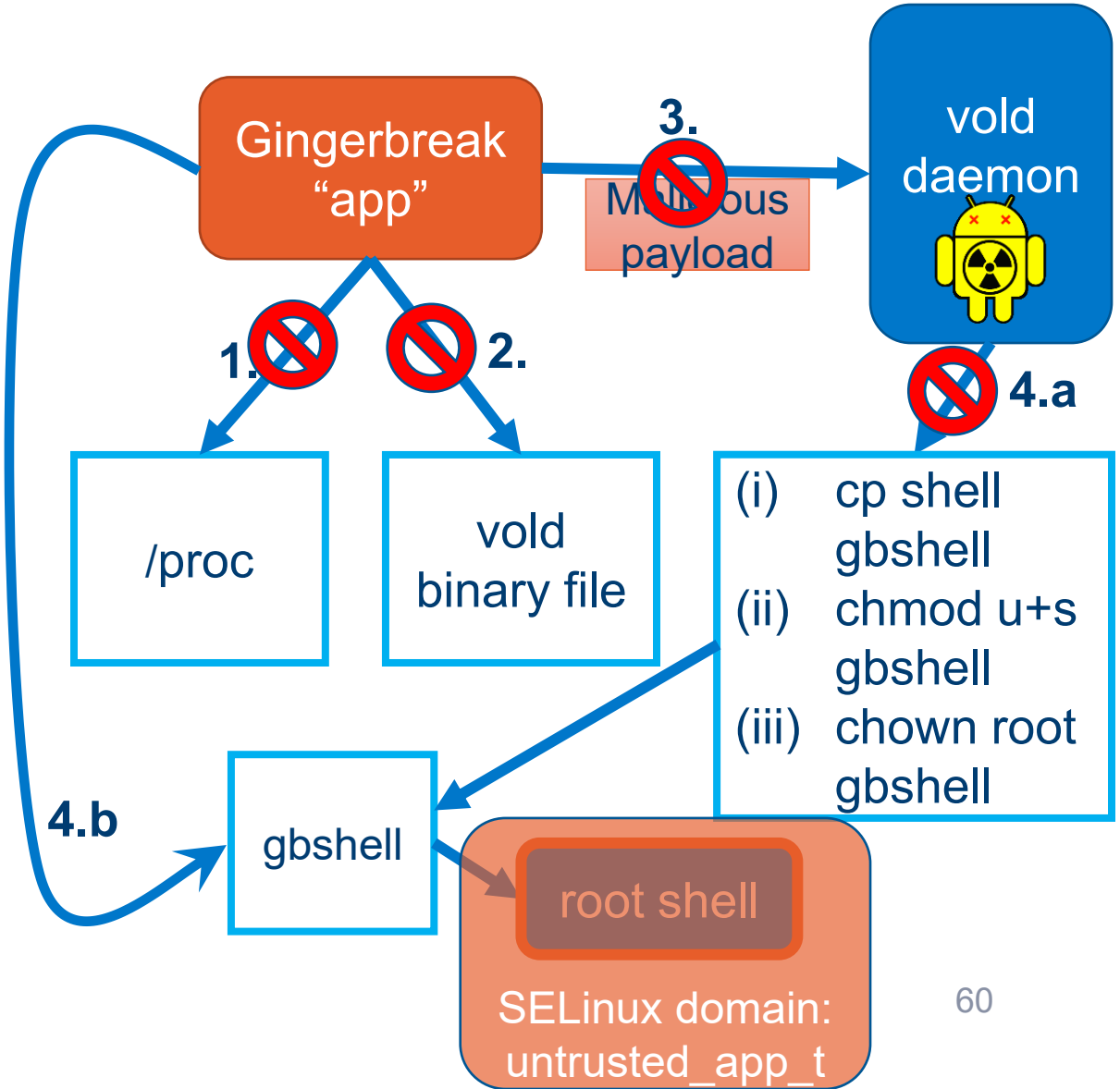
- Testers put in temporary rules to see what would happen next.
- SE Android denied setting the ownership and mode of the file (4.a) so the shell would not be setuid-root.



GingerBreak on SE Android (4/)

Testers then temporarily allowed 1,2,3, & 4.a to see the next phase.

- GingerBreak executed the setuid-root shell, which did provide the user with a uid 0 shell, but the SELinux context remained the same so it did not have superuser privs.



Application vulnerabilities - Skype

The app was storing sensitive user data without encryption in files within its data directory that are world readable and writable.

- E.g., account balance, date of birth, home address, chat logs, etc.

This meant that every other app on the device could read the data.

- If allowed internet permission, it could leak the data remotely as well.
- They could also maliciously tamper with the files.

Classic DAC example!

Skype on SE Android

Policy was configured so that no app can read or write files created by another app

- by assigning each app and its files a unique MLS category.

Result: SE Android prevents any malicious app from reading or writing the files (despite weak DAC permissions).

Opera Mobile

This is a version of the Opera web browser built for Android.

It had a vulnerability where the app created its cache files world-readable and world-writeable.

Any app on the device could both read and write to the browser's cache

- potentially leaking sensitive user information and altering data or code (e.g., Javascript).

Opera Mobile on SE Android

Policy ensures that no other app on the device could read or write the cache files of the browser.

There are other examples of this type of vulnerability due to the misuse of umask.

- See the paper [3] if interested.

Summary

- Android has lots of security features
- No one feature is perfect, but combined they are useful
- Cannot prevent the problem of users giving permissions to malicious apps and therefore their personal data
- But we can limit how these apps affect the system
- Exploits are still possible, but much more challenging for attackers to develop with SELinux being used.